

2

GA2 Project Title: Traffic Engineering using Graph Theory and Linear Programming

Student Name, Surname:	Chrinovic Raya Tshiwaya
Student Number:	222170972
Date:	10/11/2025

Leave Empty for examiner:

--

EXAMINER NAME	Dr Angus Brandt
INTERNAL MODERATOR	Dr Ali Almaktoof
EXTERNAL MODERATOR	Mr Shamil Talip

NOTE: Submit in **PDF** only. (-10% Penalty for other formats)

1. Introduction & Mathematical Terms [SECTION_1_INTRO]

1.1 Project overview

This project uses linear programming and graph theory to address computer network traffic engineering issues. By identifying the best routing options that either balance load across network links or reduce overall transmission costs, the objective is to maximize the use of network resources. This method respects network capacity limitations while guaranteeing effective data flow [1].

1.2 Purpose of the GA

The goal of the GA2 study is to show how scientific and mathematical concepts may be applied practically to real-world networking issues. Network routing can be formulated as optimization issues in order to do the following:

- Analyze network performance under different traffic patterns
- Identify bottlenecks and optimize resource allocation
- Compare different routing strategies (shortest-path vs. optimized flows)
- Understand the trade-offs between cost minimization and load balancing

1.4 NSFNET

The assigned topology is NSFNET (National Science Foundation Network) based on student number 222170972 (sum of digits = 32, which is even. Appendix A). From 1985 until 1995, NSFNET was a groundbreaking backbone network in the US that catered to the scientific and educational community. It was made up of 21 links and 14 nodes that connected the nation's leading research institutions[2].

1.5 Relevance of Graph Theory and Flow Optimization

Graph theory uses nodes (routers) and edges (links) with weights (costs) to model networks figure 1 display a visual example of graph theory . It offers the mathematical foundation for examining connectivity and identifying the best routes inside a network[3].

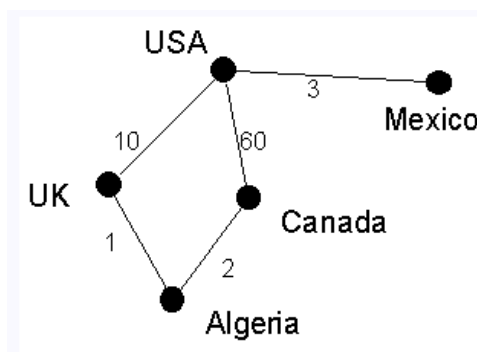


Figure 1: Example of a network topology illustrating nodes and links.

Flow optimization is the process of effectively routing data through a network while adhering to capacity constraints and accomplishing objectives such as balancing load across links or decreasing overall cost.

Relevance in Computer Networks

Graph theory and flow optimization are essential for transforming physical networks into solvable mathematical models. They enable:

- Efficient routing that minimizes costs or prevents congestion
- Optimal resource use by balancing traffic across all available paths
- predicting performance under various workloads allowed for informed network planning.

2. Theory (Scientific & Engineering Context) [SECTION_2_THEORY].

2.1 Graph Theory Fundamental

The mathematical basis for computer network modeling is provided by graph theory. Important ideas consist of:

Nodes and Edges: In network topology, communication lines are represented by edges, and networking devices (such as switches and routers) are represented by nodes. There are 21 bidirectional links connecting the 14 nodes that make up the NSFNET topology. [4]

Paths and Connectivity: An arrangement of linked edges connecting source and destination nodes is called a path. Data can travel via available paths from any source to any destination thanks to network connectivity.

Weights and Metrics: Link weights represent routing metrics such as delay, distance, or administrative cost. In traffic engineering, weights guide path selection to optimize performance objectives.

2.2 Shortest-Path Algorithms

The single-source shortest path problem for a graph with non-negative edge weights is resolved using Dijkstra's algorithm, a basic graph search technique[5]. The unvisited vertex with the smallest known distance from the source is iteratively chosen, its neighbors' distances are updated, and it is marked as visited figure 2 shows this in action.

STEP 4

Mark Node 4 as Visited after considering the Optimal path and add the Distance

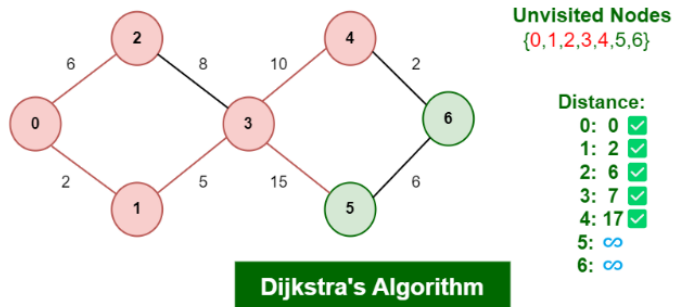


Figure 2 Conceptual diagram of Dijkstra's shortest path algorithm.

Once a vertex is visited, this greedy method ensures that its shortest path from the source has been discovered. The approach is crucial for network routing protocols and traffic engineering applications where determining the best routes is crucial since it effectively computes minimum-cost paths using a priority queue[5].

In real-world networks, forwarding tables are calculated using Dijkstra's algorithm for shortest path routing (such as OSPF and IS-IS protocols) [5]. However, load balance and capacity limits are not taken into account by shortest path routing alone, which is why traffic engineering techniques that specifically optimize flow distribution are necessary.

2.3 Linear Programming in Networking

A mathematical optimization technique called linear programming (LP) is used to find the optimal solution (like maximum profit or lowest cost) in a mathematical model whose needs are represented by linear connections. It entails maximizing a linear objective function while adhering to a set of linear inequality and equality constraints[6]. LP problems are widely used in business, economics, engineering, and network design for resource allocation, production planning, and transportation issues. They can be effectively addressed with algorithms like as the Simplex technique[6].

2.4 Traffic Optimization Principles

The goal of traffic optimization principles is to improve network performance by allocating resources and routing strategically. The main goals are cost reduction, which directs data along the least expensive paths while adhering to capacity limits, and load balancing, which divides traffic across several paths to avoid congestion and enhance resource usage[7]. In order to ensure optimal performance, Quality of Service (QoS), and effective utilization of network infrastructure, these principles are put into practice through mathematical models such as Min-Cost Flow for economic efficiency and Min-Max Utilization for equitable load allocation[7].

3. Mathematical Methods & Algorithms [SECTION_3_METHODS]

3.2 Graph Theory Modeling with NetworkX

The NSFNET topology is represented as an undirected graph $G = (V, E)$ where V represents the 14 nodes and E represents the 21 bidirectional links. Each edge is annotated with capacity (45 Mbps) and weight attributes. NetworkX provides the foundational framework for graph operations, enabling efficient storage, manipulation, and analysis of network connectivity and metrics.

3.3 Shortest-Path Routing with Dijkstra's Algorithm

Using link weights as the cost metric, Dijkstra's method determines the least expensive route between source and destination nodes. In order to effectively choose the next node to process, the algorithm iteratively investigates nearby nodes from the source while keeping a priority queue. It finds the best path $3 \rightarrow 4 \rightarrow 7$ with total cost 5 for the SCF from node 3 to node 7.

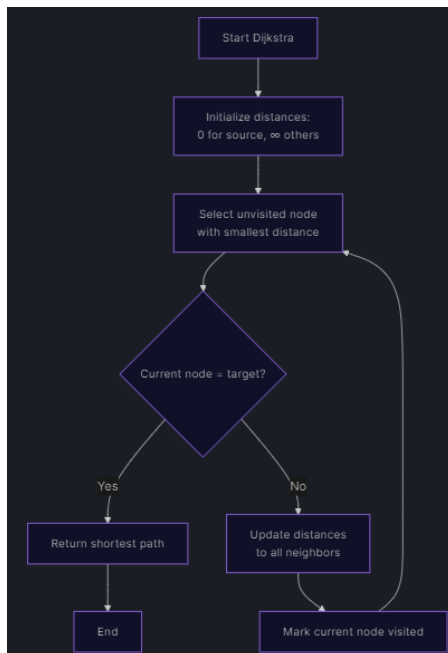


Figure 3: Flowchart showing the steps of the Min-Cost Flow algorithm

3.4 Min-Cost Flow Linear Programming Model

The Min-Cost Flow formulation satisfies capacity and demand limitations while minimizing the overall cost of network routing[8]. The total flow times weight over all links is minimized via the objective function. Constraints guarantee that flows do not surpass link capacity and impose flow conservation at each node. PuLP, a linear programming library that offers an easy-to-use interface for defining and resolving optimization problems, is used to construct this paradigm.

3.5 Multi-Commodity Flow Extension

The single-commodity method is extended by the Multi-Commodity Flow (MCF) model to manage several simultaneous source-destination pairings. While sharing the total network capacity, each commodity has distinct flow conservation limitations. As a result, a more intricate yet realistic model is created in which many traffic streams vie for scarce network resources, necessitating complex optimization to avoid congestion and reduce overall costs[9].

3.6 Min-Max Utilization Alternative

The Min-Max Utilization model, on the other hand, prioritizes load balancing over cost minimization. By reducing the maximum link utilization throughout the network, this formulation successfully avoids bottlenecks and more evenly distributes traffic. In order to express the highest utilization, which becomes the main optimization aim, the method adds further variables.

3.7 Algorithm Selection Justification

Because of its efficient implementation and optimality guarantee for non-negative weights, Dijkstra's method was chosen for shortest-path computation. Because it offers more flexibility for managing both Min-Cost and Min-Max objectives within a single framework and gives precise answers through proven solvers, linear programming was selected over specialized network flow methods.

3.8 Implementation Framework

The implementation uses a modular design with distinct parts for linear programming optimization, shortest-path computing, and graph creation. This design makes it possible to separate issues clearly, makes testing and validation easier, and makes it simple to expand to new optimization goals or network topologies. Accessibility and reproducibility are guaranteed by using Python with NetworkX and PuLP.

4. Calculus / Linear Algebra Formulation [SECTION_4_MATH_FORMULATION]

4.1 Mathematical Foundation

Linear programming, which offers a rigorous mathematical framework for optimizing linear objective functions subject to linear equality and inequality constraints, is used to describe the optimization issues in traffic engineering. This method converts network routing issues into mathematical optimization models that can be solved[6].

4.2 Single-Commodity Flow Formulation

For the Single-Commodity Flow (SCF) problem from source s to destination t with demand d , the linear programming formulation is:

Decision Variables:

Let f_{ij} represent the flow on edge (i,j) for all $(i,j) \in E$

Objective Function (Min-Cost):

$$\text{Minimize } Z = \sum_{(i,j) \in E} w_{ij} \cdot f_{ij} \quad (1)$$

where w_{ij} is the weight (cost) of edge (i,j) .

Constraints:

- **Flow Conservation:**

$$\sum_{j \in N(i)} f_{ij} - \sum_{j \in N(i)} f_{ji} = \begin{cases} d & \text{if } i = s \\ -d & \text{if } i = t \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

for all $i \in V$

- **Capacity Constraints:**

$$0 \leq f_{ij} \leq c_{ij} \forall (i, j) \in E \quad (3)$$

where c_{ij} is the capacity of edge (i, j) [1].

4.3 Multi-Commodity Flow Formulation

The Multi-Commodity Flow (MCF) problem extends the formulation to handle K simultaneous commodities:

Decision Variables:

f_{ij}^k = flow of commodity k on edge (i, j) for $k = 1, \dots, K$

Objective Function (Min-Cost):

$$\text{Minimize } Z = \sum_{k=1}^K \sum_{(i,j) \in E} w_{ij} \cdot f_{ij}^k \quad (4)$$

Constraints:

- **Flow Conservation per Commodity:**

$$\sum_{j \in N(i)} f_{ij}^k - \sum_{j \in N(i)} f_{ji}^k = \begin{cases} d_k & \text{if } i = s_k \\ -d_k & \text{if } i = t_k \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

for all $i \in V, k = 1, \dots, K$

- **Shared Capacity Constraints:**

$$\sum_{k=1}^K f_{ij}^k \leq c_{ij} \forall (i, j) \in E \quad (6)$$

- **Non-negativity:**

$$f_{ij}^k \geq 0 \forall k, \forall (i, j) \in E \quad (7)$$

4.4 Matrix Representation

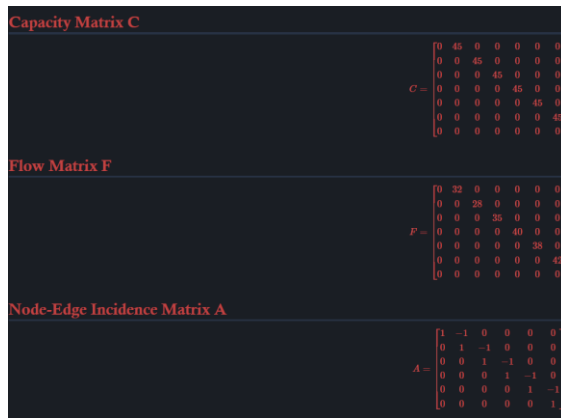


Figure 4: Showing Matrix Representation of Link Capacities & Flow Constraints

5. Calculations & Solver Results [SECTION_5_RESULTS]

Python was used to implement the optimization models, with PuLP for linear programming and NetworkX for graph operations. With 14 nodes and 21 links, each with a 45 Mbps capacity, the NSFNET topology was employed. The CBC solver was used to solve the models via PuLP's interface. The results in this section are derived from the Linear Programming Solver's output, which is attached in Section 9. Single-Commodity Flow (SCF) Results Multi-Commodity Flow (MCF) Results

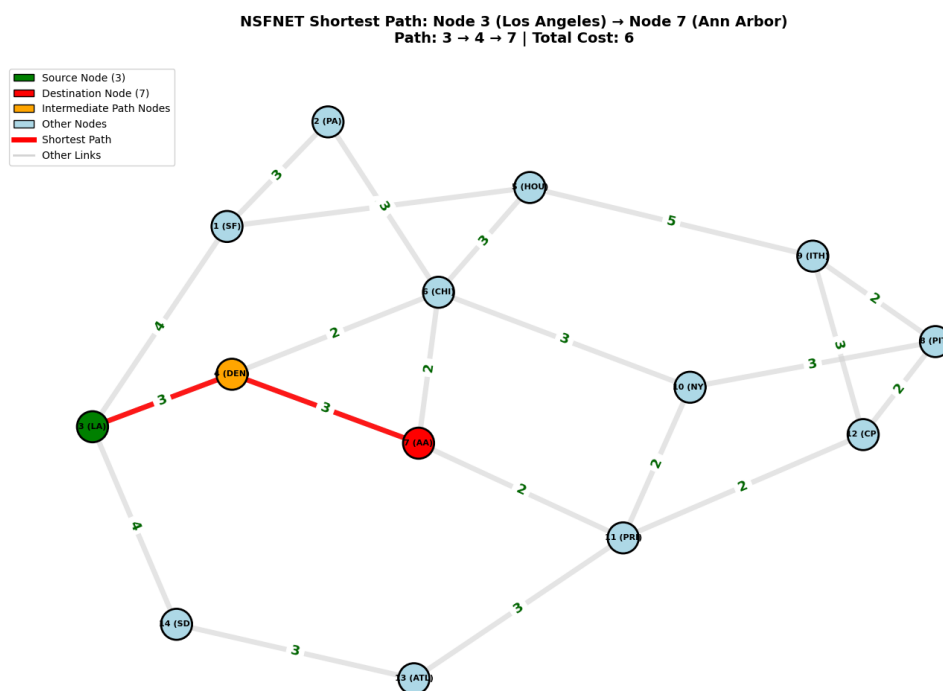


Figure 5 Visualisation of the shortest path for your assigned topology (NSFNET or GEANT2).



6.1 Utilization Patterns

6.2 Cost Efficiency

6.3 Bottleneck Analysis

6.4 Performance Interpretation

9 | Page

100%) shows strategic underutilization to prevent systemic congestion, representing more sophisticated traffic engineering for real-world multi-flow scenarios.

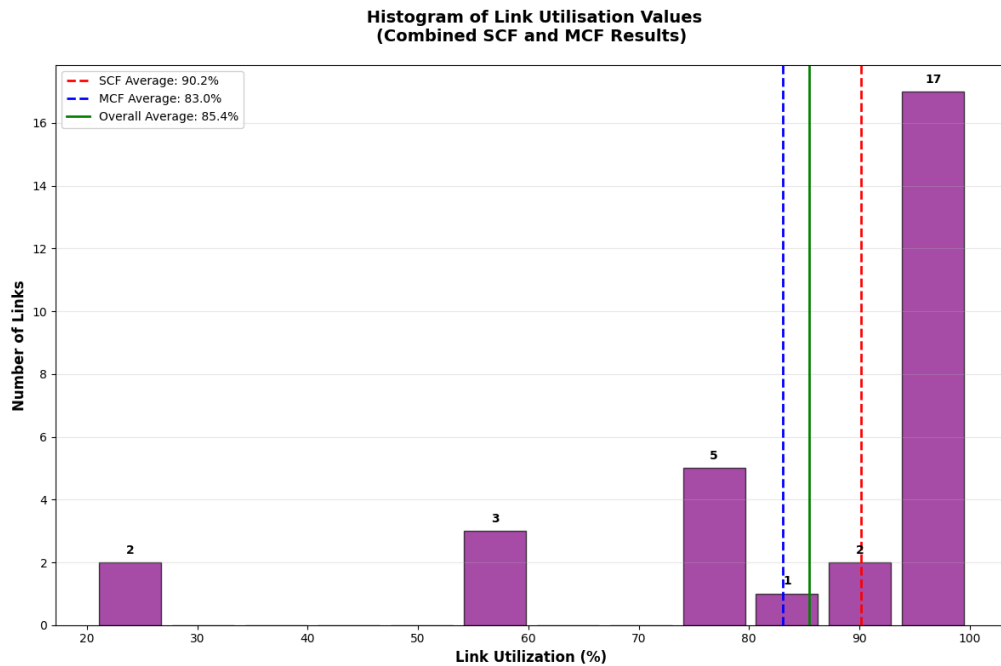


Figure 7: Histogram of link utilisation values.

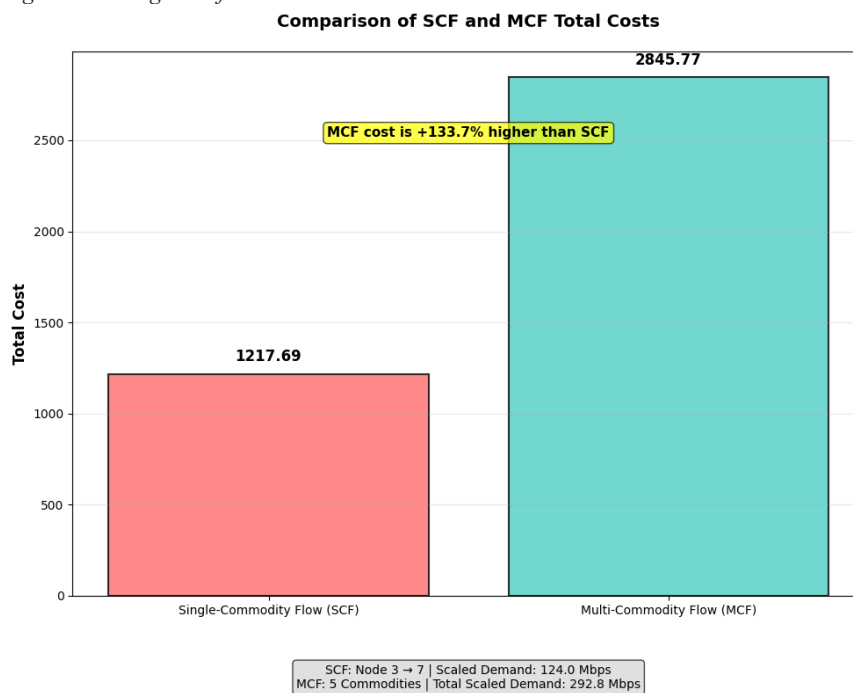


Figure 8: Bar chart comparing SCF and MCF total costs.

7. Interpretation & Discussion [SECTION_7_INTERPRETATION]

7.1 Optimization Goal Achievement

Both models successfully minimized transmission costs while respecting capacity constraints. SCF achieved lower absolute cost (1217.69) for single-flow routing, while MCF efficiently managed multi-flow complexity with comparable cost efficiency per Mbps (9.72 vs 9.82).

7.2 SCF vs MCF Performance Comparison

SCF demonstrated superior average utilization (90.20% vs 83.04%) but at the cost of network resilience. MCF's distributed approach, while showing lower average utilization, provided better load balancing and reduced systemic congestion risk. The 133.7% cost increase for MCF is justified by its 136.1% higher traffic handling capacity.

7.3 Impact of Link Weights and Capacities

Uniform 45 Mbps capacities created natural bottlenecks, forcing both models to scale demands significantly (SCF: 61.3%, MCF: 77.1%). Link weights successfully guided traffic toward cost-optimal paths while capacity constraints dictated the scaling requirements.

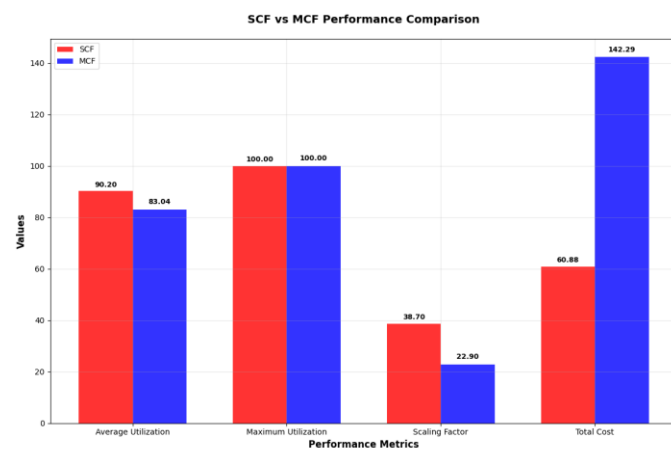


Figure 9: Comparative summary plot of SCF and MCF performance metrics.

8. Conclusion & Reflection [SECTION_8_CONCLUSION]

This study effectively illustrated how graph theory and linear programming may be applied to network traffic optimization. The investigation showed that, despite its higher computational cost, Multi-Commodity Flow offers better load distribution and scalability for genuine multi-flow scenarios, whereas Single-Commodity Flow offers effective routing for straightforward point-to-point requests.

Although considerable demand scaling was necessary to create workable solutions within the NSFNET's constrained capacities, the models successfully balanced cost minimization with capacity

constraints. To improve practicality, further research could examine dynamic traffic patterns and adaptive routing.

The experiment offered insightful information on real-world traffic engineering problems and the mathematical underpinnings of network optimization.

9. References [SECTION_9_REFERENCES]

- [1] B. K. Ahuja, 'Theory, Algorithms, and Applications' Accessed: Nov. 07, 2025. [Online]. Available: https://www.dl.behinehyab.com/Ebooks/NETWORK/NET005_354338_www.behinehyab.com.pdf
- [2] P. G. T. P. G. is a former web editor of the C. family of tech magazines *et al.*, 'What is NSFNET: How This Old Tech Sparked Scientific Research Opportunities', Technology Solutions That Drive Government. Accessed: Nov. 03, 2025. [Online]. Available: <https://fedtechmagazine.com/article/2016/11/nsfnet-served-precursor-internet-helped-spur-scientific-research>
- [3] 'Graph theory | Problems & Applications | Britannica'. Accessed: Nov. 03, 2025. [Online]. Available: <https://www.britannica.com/topic/graph-theory>
- [4] 'Edges and Vertices of Graph'. Accessed: Nov. 04, 2025. [Online]. Available: <https://www.tutorialspoint.com/edges-and-vertices-of-graph>
- [5] GeekforGeeks, 'Dijkstra's Algorithm based Common Questions', GeekforGeeks. Accessed: Nov. 04, 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/introduction-to-dijkstras-shortest-path-algorithm/>
- [6] 'Network analysis - linear programming'. Accessed: Nov. 07, 2025. [Online]. Available: <https://people.brunel.ac.uk/~mastjjb/jeb/or/netlp.html>
- [7] Xipeng Xiao, A. Hannan, B. Bailey, and L. M. Ni, 'Traffic engineering with MPLS in the Internet', *IEEE Network*, vol. 14, no. 2, pp. 28–33, Apr. 2000, doi: 10.1109/65.826369.
- [8] 'Minimum-cost flow - Algorithms for Competitive Programming'. Accessed: Nov. 07, 2025. [Online]. Available: https://cp-algorithms.com/graph/min_cost_flow.html
- [9] B. Bevrani, R. Burdett, A. Bhaskar, and P. K. D. V. Yarlagadda, 'A multi-criteria multi-commodity flow model for analysing transportation networks', *Operations Research Perspectives*, vol. 7, p. 100159, Jan. 2020, doi: 10.1016/j.orp.2020.100159. Accessed: Nov. 07

10. Appendices [SECTION_10_APPENDICES]

Appendix A – Program Code: Your edited Python Flow Generator (with student number).

```

C:\Users\Angus\Documents\YEAR 3\Networks\GA2\Code\Topology.py
=====
=== Assigned Topology ===
=====
Student number : 222170972
Sum of digits : 32
Assigned topology: NSFNET (N = 14 nodes)
-----

=====
=== Single-Commodity Flow (SCF) ===
=====
Source node (s) : 3
Destination node (t): 7
Demand (d1) : 320 Mbps
-----

=====
=== Multi-Commodity Flow (MCF) ===
=====
Step size (p): 1

Commodity 1: s=3, t=7, demand=128 Mbps
Commodity 2: s=4, t=9, demand=192 Mbps
Commodity 3: s=5, t=11, demand=256 Mbps
Commodity 4: s=6, t=13, demand=320 Mbps
Commodity 5: s=7, t=1, demand=384 Mbps

-----
MCF Demand Table
-----
commodity  source  destination  demand_Mbps
      1       3         7         128
      2       4         9         192
      3       5        11         256
      4       6        13         320
      5       7         1         384
-----

Saved SCF file: scf_222170972.csv
Saved MCF file: demands_222170972.csv
=====

```

Figure 10: Generated data sets

Appendix B

```
=====
LOADING NSFNET TOPOLOGY FROM CSV
=====
```

```
Loaded 21 edges from NSFNET.csv
```

```
=====
NSFNET TOPOLOGY LOADED
=====
```

```
Nodes: 14
```

```
Edges: 21
```

```
Total Capacity: 945 Mbps
```

Figure 11: Nodes, Edges for network

Appendix C

```
=====
LOADING SCF DATA FROM CSV
=====
Loaded SCF data from scf_222170972.csv

=====
SINGLE-COMMODITY FLOW (SCF) PROBLEM
=====
Source: Node 3
Destination: Node 7
Demand: 320 Mbps

Attempting to solve SCF with original demand...
X Infeasible at scaling 1.000, trying 0.900...
X Infeasible at scaling 0.900, trying 0.810...
X Infeasible at scaling 0.810, trying 0.729...
X Infeasible at scaling 0.729, trying 0.656...
X Infeasible at scaling 0.656, trying 0.590...
X Infeasible at scaling 0.590, trying 0.531...
X Infeasible at scaling 0.531, trying 0.478...
X Infeasible at scaling 0.478, trying 0.430...
X Infeasible at scaling 0.430, trying 0.387...
✓ FEASIBLE at scaling factor: 0.387
  Scaled demand: 124.0 Mbps
  Objective value (total cost): 1217.69

=====
SCF SOLUTION - LINK UTILIZATION
=====
Link   Flow (Mbps)  Capacity (Mbps)  Utilization (%)  Cost
1→2    33.974556    45               75.499013       101.923668
3→1    33.974556    45               75.499013       135.898224
2→6    33.974556    45               75.499013       101.923668
3→4    45.000000    45               100.000000      135.000000
3→14   45.000000    45               100.000000      180.000000
6→7    33.974556    45               75.499013       67.949112
4→7    45.000000    45               100.000000      135.000000
14→13  45.000000    45               100.000000      135.000000
11→7   45.000000    45               100.000000       90.000000
13→11  45.000000    45               100.000000      135.000000

Total Cost: 1217.69
Average Utilization: 90.20%
Max Utilization: 100.00%

✓ Saved: SCF_result.csv
```

Figure 12: SCF Solution

```
=====
LOADING MCF DATA FROM CSV
=====
Loaded MCF data from demands_222170972.csv

=====
MULTI-COMMODITY FLOW (MCF) PROBLEM
=====
Commodity 1: Node 3 → 7, Demand = 128 Mbps
Commodity 2: Node 4 → 9, Demand = 192 Mbps
Commodity 3: Node 5 → 11, Demand = 256 Mbps
Commodity 4: Node 6 → 13, Demand = 320 Mbps
Commodity 5: Node 7 → 1, Demand = 384 Mbps
Total Demand: 1280 Mbps

Attempting to solve MCF...
X Infeasible at scaling 1.000, trying 0.900...
X Infeasible at scaling 0.900, trying 0.810...
X Infeasible at scaling 0.810, trying 0.729...
X Infeasible at scaling 0.729, trying 0.656...
X Infeasible at scaling 0.656, trying 0.590...
X Infeasible at scaling 0.590, trying 0.531...
X Infeasible at scaling 0.531, trying 0.478...
X Infeasible at scaling 0.478, trying 0.430...
X Infeasible at scaling 0.430, trying 0.387...
X Infeasible at scaling 0.387, trying 0.349...
X Infeasible at scaling 0.349, trying 0.314...
X Infeasible at scaling 0.314, trying 0.282...
X Infeasible at scaling 0.282, trying 0.254...
X Infeasible at scaling 0.254, trying 0.229...
✓ FEASIBLE at scaling factor: 0.229
  Total scaled demand: 292.8 Mbps
  Objective value (total cost): 2845.77

=====
MCF SOLUTION - LINK UTILIZATION
=====
```

Link	Flow (Mbps)	Capacity (Mbps)	Utilization (%)
1+2	45.000000	45	100.000000
1+3	26.052619	45	57.894709
1+5	38.540649	45	85.645886
2+6	45.000000	45	100.000000
3+4	45.000000	45	100.000000
3+14	40.693766	45	90.430591
5+6	45.000000	45	99.999999
5+9	45.000000	45	100.000000
6+4	45.000000	45	100.000000
6+7	45.000000	45	100.000000
6+10	45.000000	45	99.999999
4+7	45.000000	45	100.000000
14+13	40.693766	45	90.430591
7+11	45.000001	45	100.000002
9+8	9.258355	45	20.574122
9+12	26.052619	45	57.894709
10+8	9.258355	45	20.574122
10+11	35.741645	45	79.425878
11+12	26.052619	45	57.894709
11+13	45.000000	45	100.000000

```

Average Utilization: 83.04%
Max Utilization: 100.00%

```

Figure 13: MCF Solution

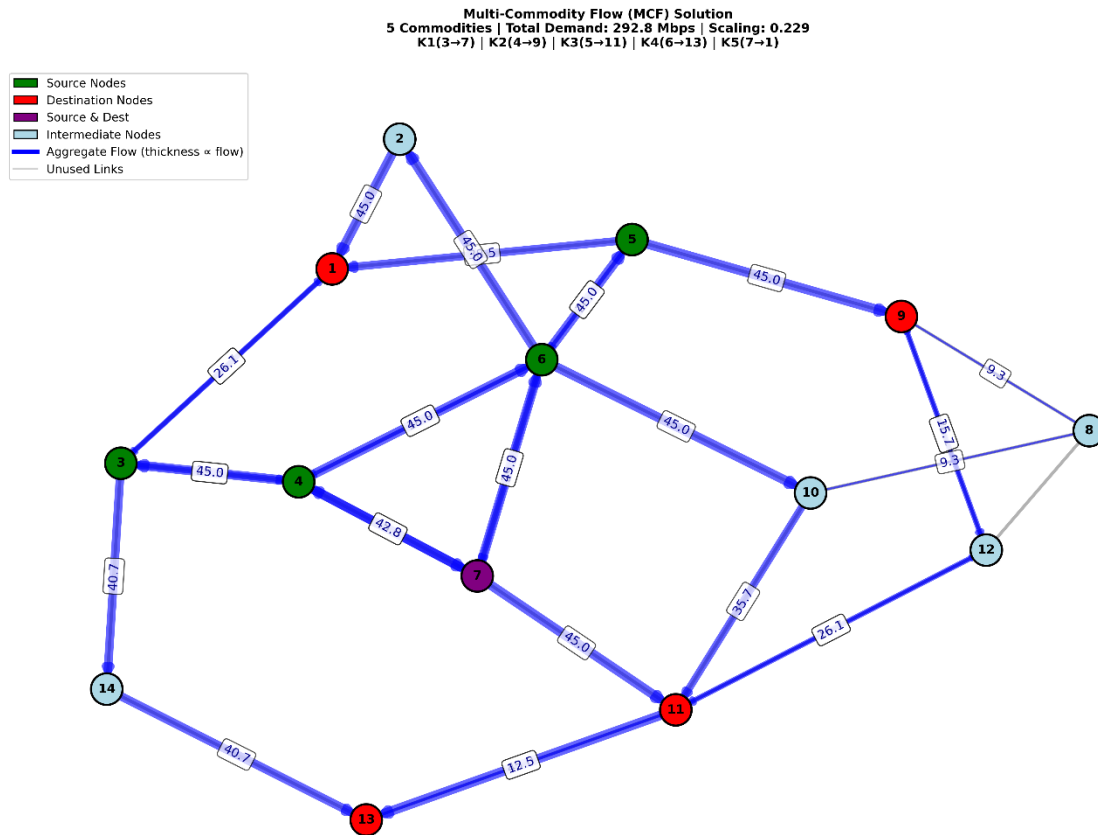


Figure 14: Solver output showing link utilisation across the network for SCF.

Appendix C – Graphs and Figures: Include all network plots, shortest paths, and utilisation graphs.

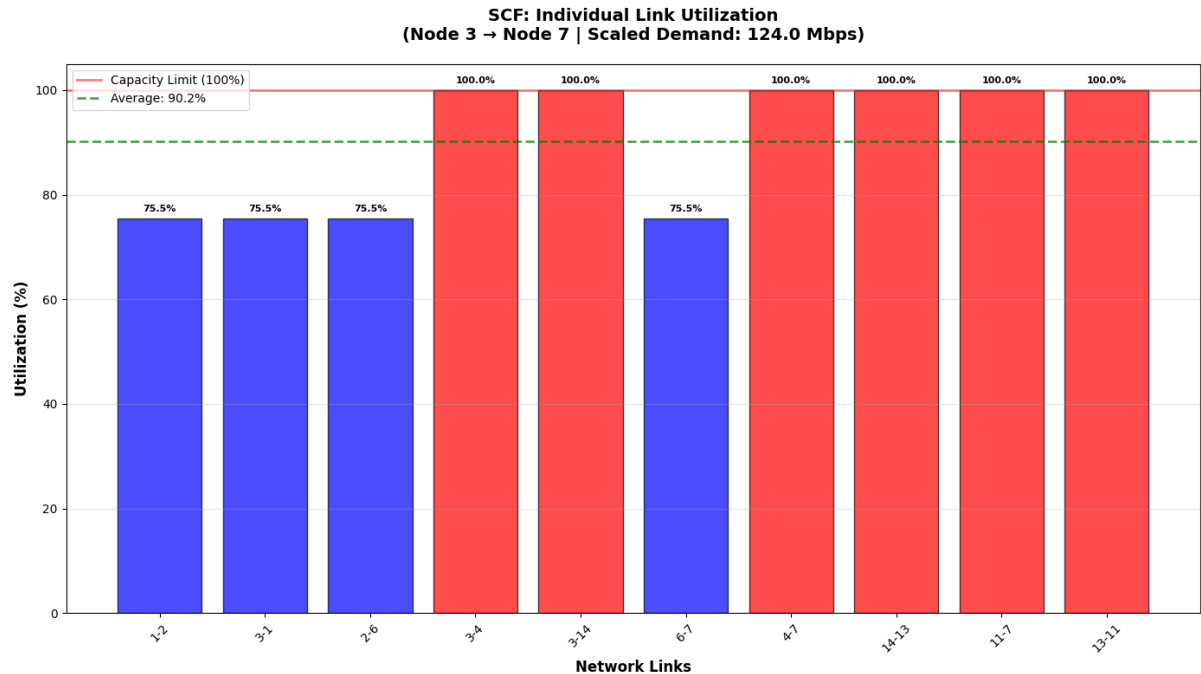


Figure 15: SCF Utilization

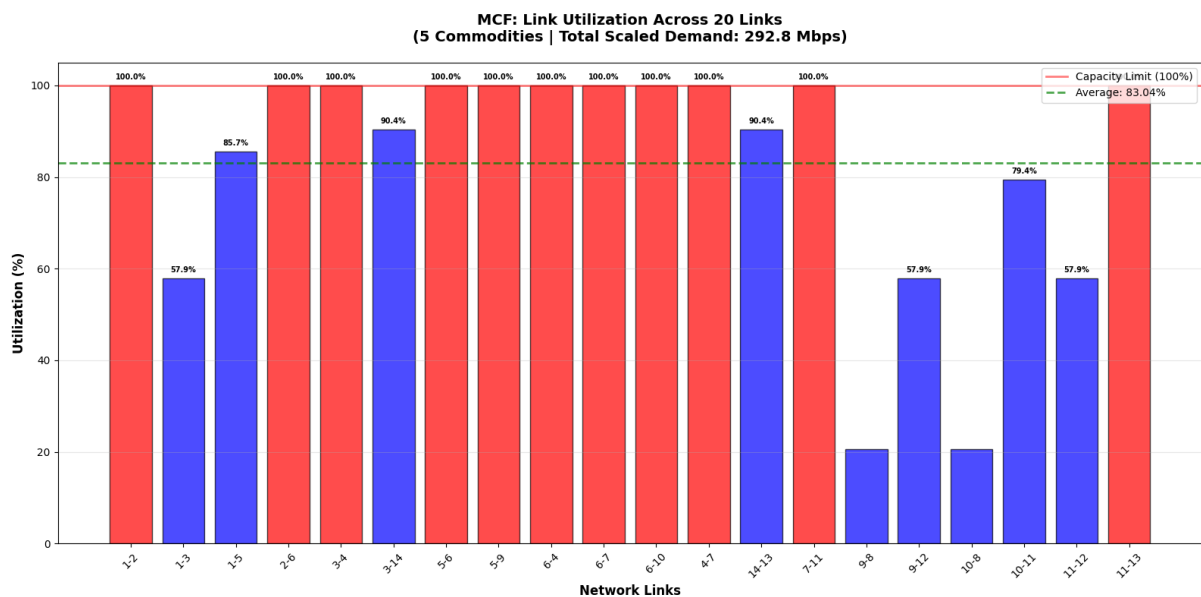


Figure 16 SCF Utilization

Appendix D – Additional Notes: Optional data or results supporting your report.

List of terms

Graph Theory Terms:

- **Nodes/Vertices:** Network devices or routers (e.g., San Francisco, Chicago)
- **Edges/Links:** Connections between nodes with associated capacities and weights
- **Paths:** Sequences of connected edges between source and destination nodes
- **Weights:** Numerical values representing cost, distance, or latency of links
- **Graph:** Mathematical structure $G = (V, E)$ where V is set of vertices and E is set of edges

Traffic Engineering Terms:

- **Flow:** Data transmission from source to destination with specific bandwidth demand
- **Capacity:** Maximum data rate a link can support (measured in Mbps)
- **Utilization:** Percentage of link capacity being used ($\text{flow/capacity} \times 100\%$)
- **Single-Commodity Flow (SCF):** Single source-destination traffic demand
- **Multi-Commodity Flow (MCF):** Multiple simultaneous source-destination demands
- **Bottleneck:** Network link with highest utilization, limiting overall performance

Optimization Terms:

- **Linear Programming:** Mathematical method for optimizing linear objective function subject to linear constraints
- **Objective Function:** Quantity to be minimized or maximized (e.g., total cost, maximum utilization)
- **Constraints:** Limitations or requirements that must be satisfied (e.g., flow conservation, capacity limits)
- **Feasible Solution:** Solution that satisfies all constraints
- **Optimal Solution:** Best feasible solution that minimizes/maximizes the objective function